

Word-in-Context

IN5550 – Obligatory Assignment 2

Sigurd Nordbye
Ola Skeie
Zeeshan Shahid

Department of Informatics
University of Oslo

Spring 2026

1 Task A

1.1 A1: Static Embedding Baseline

1.1.1 Method

As a baseline, we use pretrained static Word2Vec embeddings (NLPL model 40, 100 dimensions) to represent the target word in context. For each sentence, we extract the embeddings of the target word and the $N = 3$ surrounding words on each side, and average them into a single 100-dimensional vector. The two resulting vectors $\mathbf{v}_a$ and $\mathbf{v}_b$ are then combined as $[\mathbf{v}_a; \mathbf{v}_b; |\mathbf{v}_a - \mathbf{v}_b|]$, giving a 300-dimensional input to a two-layer MLP with hidden size 256, ReLU activations, and dropout of 0.3. The model is trained with cross-entropy loss and AdamW (lr=1e-3) for 10 epochs.

1.1.2 Results

Epoch	Krippendorff's α	Accuracy
1	0.0663	0.3072
2	0.0889	0.3106
3	0.1535	0.3508
4	0.1164	0.3520
5	0.1836	0.3538
6	0.2113	0.3698
7	0.1912	0.3184
8	0.1781	0.3644
9	0.1907	0.3429
10	0.1936	0.3635

Table 1: Static embedding baseline performance on the English validation set per epoch.

The model achieves a best Krippendorff's α of 0.2113 and accuracy of 36.98% at epoch 6, after which performance fluctuates without clear improvement. Accuracy of 36% is only marginally better than random guessing (25%), and α below 0.2 indicates near-chance agreement with gold labels.

This is expected: static embeddings assign a single fixed vector to each word regardless of context. The word *arm* receives the same representation whether it refers to a body part or a weapon. The model therefore cannot distinguish between homonymous or polysemous usages, which is precisely what the WiC task requires. Contextualized models such as BERT address this by computing word representations dynamically based on the surrounding sentence.

1.2 A2: Language Model Comparison

1.2.1 Models

We compare two pretrained transformer models: BERT and mmBERT . BERT is trained on English text only, while mmBERT is trained on many languages and is therefore more suitable for multilingual settings.

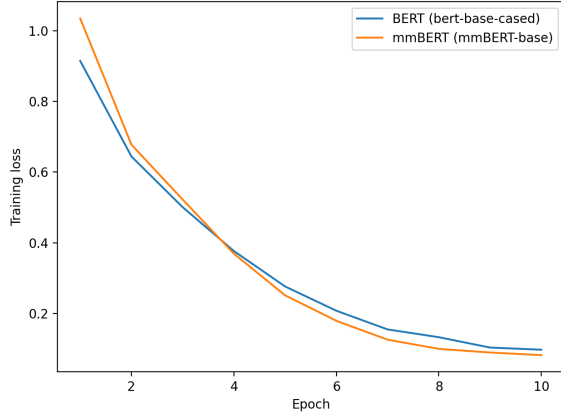


Figure 1: Training loss per epoch for BERT and mmBERT.

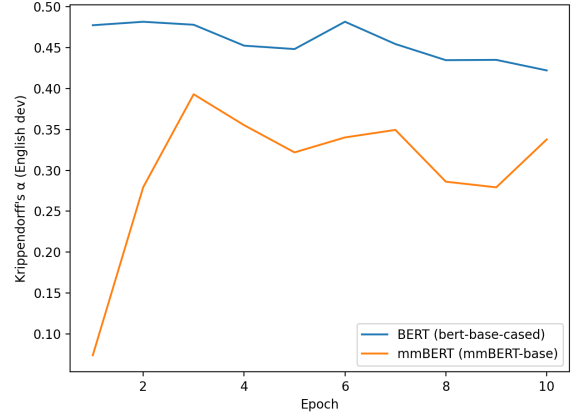


Figure 2: Validation Krippendorff's α per epoch for BERT and mmBERT.

1.2.2 Analysis and observed differences

Figure 1 shows the training loss for both models across epochs. For both BERT and mmBERT, the training loss decreases steadily throughout training, indicating that both models converge successfully. The curves are smooth and consistently decreasing.

Figure 2 shows the validation performance measured by Krippendorff's alpha on the English dev set. Here a clearer difference between the models can be observed.

BERT maintains relatively stable validation performance across epochs, with scores consistently around 0.45–0.48. In contrast, mmBERT shows more variation between epochs. Its score increases quickly during the first few epochs but then fluctuates throughout training and remains consistently below the BERT scores.

Although mmBERT achieves slightly lower training loss, it does not achieve better validation performance. This suggests that while the model fits the training data well, it generalizes less effectively to the validation data compared to BERT.

1.3 Explanation based on model differences

The results can be explained by the differences in how the models are trained. BERT is pre-trained solely on English data, including BooksCorpus and English Wikipedia (Devlin et al., 2019), while mmBERT is trained on over 1800 languages (Marone et al., 2025). Because BERT is trained only on English, the model can learn more detailed and specialized representations

of English vocabulary, grammar, and context. In contrast, mmBERT must distribute its model capacity across many languages, meaning that the same parameters and vocabulary are shared between them. As a result, the model cannot specialize as strongly for English as a monolingual model like BERT. This can lead to slightly lower performance and more variability when evaluating on an English-only dataset, which may explain why BERT produced more stable results and higher Krippendorff’s alpha in our experiments.

1.4 A3: Output Pooling

1.4.1 Method

We compare two pooling strategies using BERT and mmBERT. BERT is a monolingual English model, while mmBERT is a multilingual variant, making it relevant for cross-lingual transfer in Task B. For each configuration we finetune the model for 10 epochs on the English training set and evaluate on the English validation set.

1.4.2 Pooling Strategies

We compare two strategies for extracting representations from the encoder.

BOS pooling uses the representation of the [CLS] token, which is always the first token in the sequence. Since BERT processes the full sentence pair as a single sequence, the [CLS] token attends to all tokens in both sentences and accumulates a global representation of the input. This is the standard approach for sentence-level classification tasks.

Target-word pooling explicitly extracts the contextualized embeddings of the target word in each sentence. Since both models use subword tokenization, the target word may span multiple tokens. We identify these using the character-level `word_indices` provided in the dataset and average their embeddings. This gives two vectors $\mathbf{v}_a$ and $\mathbf{v}_b$, one for the target word in each sentence, which are concatenated as $[\mathbf{v}_a; \mathbf{v}_b]$ and passed to a linear classifier. We also experimented with appending the absolute difference $|\mathbf{v}_a - \mathbf{v}_b|$ as an explicit similarity signal, giving a $[\mathbf{v}_a; \mathbf{v}_b; |\mathbf{v}_a - \mathbf{v}_b|]$ representation. This did not improve performance and was left out of the final configuration.

1.4.3 Results

Model	Pooling	Best Krippendorff’s α	Best Accuracy
BERT	BOS	0.4530	0.4699
BERT	Target-word	0.5357	0.5132
mmBERT	Target-word	0.5612	0.4433

Table 2: Best validation performance per model and pooling strategy on the English validation set.

Target-word pooling outperforms BOS pooling across both models, confirming that explicitly comparing the target word’s contextualized representation in each sentence is more effective than relying on the [CLS] token. mmBERT with target-word pooling achieves the best overall α of 0.5612, and is therefore used as the base model for Task B where cross-lingual transfer to Norwegian is required.

1.5 A4: Loss Functions

1.5.1 Setup

In this experiment, we compared different loss functions while keeping the rest of the setup unchanged. All models used mmBERT with BOS/CLS pooling and a linear classification head over the four labels. Each run was trained for 10 epochs on the English training set and evaluated on the English validation set, using AdamW, a learning rate of $2e-5$, and batch size 32.

1.5.2 Results

Loss	Best Krippendorff’s α	Best Accuracy
CE (median)	0.3930	0.4448
MSE (median)	0.4744	0.3877
Squared EMD (median)	0.4295	0.4351
Squared EMD (full dist.)	0.4516	0.4905

Table 3: Best validation performance for each loss function.

The ranking only partly matched our expectations. Since the labels form an ordered scale from 1 to 4, we expected the losses that take this ordering into account to do better than standard cross-entropy. This was mostly true: squared EMD on median labels performed better than cross-entropy, and the full-distribution version gave the highest accuracy. What was more surprising was that MSE achieved the best Krippendorff’s α . MSE still uses the order of the labels, because predicting 3 instead of 4 is treated as a smaller error than predicting 1 instead of 4, but it is still a simpler regression approach and does not model a full class distribution. Because of that, we had expected the EMD-based losses, especially the full-distribution version, to perform best on the primary metric, since they are more directly designed for ordinal labels and can also capture annotator disagreement.

1.5.3 Discussion

Why might MSE be suboptimal despite also encoding order?

MSE can be suboptimal even though it takes the ordering of the labels into account, because the model only learns to predict a single numeric value rather than a probability distribution over the ordered classes. By framing the task as regression and training on the median label,

information about annotator disagreement is lost, and the model cannot represent cases where multiple labels are plausible at the same time. Another weakness is that MSE assumes equal distances between the labels, treating the difference between 1 and 2 the same as the difference between 3 and 4, even though ordinal labels guarantee order but are not necessarily equally spaced in a semantic sense. For example, the difference between labels 1 and 2 may represent a larger change in meaning than the difference between 3 and 4.

What does the EMD loss gain from respecting the ordinal structure?

The main advantage of EMD is that it penalizes predictions based on how far they are from the correct label, instead of treating all wrong predictions as equally wrong. For example, if the true label is 4, predicting 3 should be considered a smaller mistake than predicting 1. EMD compares the cumulative distributions of the predicted and target distributions, so the loss is smaller when the model places probability near the correct label and larger when it places probability farther away. This fits the task especially well because each instance is annotated by multiple people, meaning that the target can be a full distribution rather than a single label.

What additional information does the full distribution provide compared to the median?

The full distribution provides more information than the median because it preserves how the annotators were split across the four labels, not just the single middle value. This means it captures both agreement and disagreement in the data. For example, if the labels are [3,3,3,4,4], the median is 3, but that hides the fact that a substantial number of annotators chose 4. Using the full distribution instead retains this uncertainty by representing the target as a soft distribution rather than a single hard label. The full distribution gives the model a more realistic training signal than the median alone.

2 Task B: Cross-lingual Transfer

2.1 Best Configuration from Task A

Based on the results from Task A, we use mmBERT with target-word pooling as our base model for all cross-lingual transfer experiments. This configuration achieved the best Krippendorff’s α of 0.5612 on the English validation set. mmBERT is particularly suited for cross-lingual transfer as it was pretrained on many languages, including all languages in our dataset. We train with EMD loss, learning rate $1e-5$, batch size 16, and 10 epochs. During training we observed that the model tends to overfit after epoch 3, with validation alpha peaking early and declining or stagnating in later epochs. We therefore track the best validation alpha across all epochs and report results from the best checkpoint rather than the final epoch.

2.2 B1: Transfer Based on Language Similarity

2.2.1 Experimental Setup

We train four separate models on English, German, Spanish, and Chinese respectively, keeping all other hyperparameters fixed. After training, each model is evaluated on the validation sets of all six languages including Norwegian, which has no training data.

Before running experiments, we performed a brief data analysis to better understand the differences between languages. Figure 3 shows average sentence pair length, dataset size, and label distribution per language.

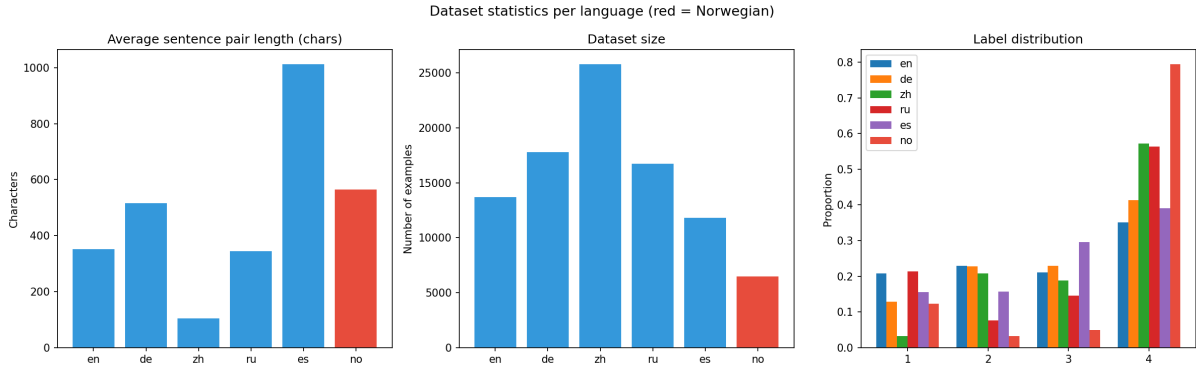


Figure 3: Dataset statistics per language. Norwegian (red) is validation-only.

Norwegian has a heavily skewed label distribution towards label 4 (identity), which is notably different from the other languages. This suggests that Norwegian annotators rated most word pairs as semantically identical, which may affect how well models trained on other languages transfer to Norwegian.

German presented practical challenges during training due to exceptionally long sentences — the maximum sentence pair length in the German training set is 10,463 characters, compared to 1,528 for English. This caused GPU out-of-memory errors, which we resolved by setting `max_length=256` tokens during tokenization. Only 0.1% of German examples exceed 7,500 characters, so the impact of truncation is minimal.

Language	Avg. length (chars)	Max length (chars)	Dataset size	% over 1000 chars
English	352	1,528	13,685	0.5%
German	516	10,463	17,791	8.1%
Chinese	105	402	25,805	0.0%
Russian	345	3,069	16,749	1.1%
Spanish	1013	3,938	11,826	46.3%

Table 4: Training set statistics per language.

After training, each model is evaluated on the validation sets of all six languages, English, Ger-

man, Chinese, Russian, Spanish, and Norwegian. Note that Russian is included as an additional evaluation language despite not being used for training, to give a broader picture of cross-lingual transfer.

2.2.2 Results

Train lang	en	de	zh	ru	es	no
English	0.5459	0.1492	0.4678	0.2365	0.5007	0.4340
German	0.5941	0.6240	0.1809	0.4043	0.4976	0.4356
Chinese	0.3536	0.2176	0.4723	0.1882	0.2597	0.3659
Spanish	0.4011	0.4930	0.0661	0.4051	0.5912	0.3340

Table 5: Krippendorff’s α on all validation sets per training language. Norwegian column in bold.

Training on German yields the best transfer to Norwegian ($\alpha = 0.4356$), closely followed by English ($\alpha = 0.4340$). This aligns with linguistic expectations: Norwegian, German, and English are all Germanic languages and share substantial vocabulary and syntactic structure. Spanish and Chinese give weaker Norwegian performance ($\alpha = 0.3340$ and $\alpha = 0.3659$ respectively), with Spanish performing worst despite being a European language. This is somewhat surprising, but may reflect differences in label distribution and dataset composition rather than linguistic distance alone.

2.3 B2: Custom Language Selection

2.3.1 Motivation and Strategy

Based on the B1 results, we observe that German and English give the best transfer to Norwegian. This aligns with prior work showing that cross-lingual transfer performance correlates with linguistic similarity between source and target language [1]. Norwegian, German, and English all belong to the Germanic language family and share common ancestry in Proto-Germanic [2]. Norwegian is a North Germanic language, while German and English are West Germanic, meaning they are not as close to Norwegian as Swedish or Danish would be, but still share substantial vocabulary and syntactic structure.

Based on these observations, we design two multilingual training sets:

1. **de + en**: The two best-performing languages from B1, both Germanic.
2. **de + en + ru**: Adding Russian as a large Indo-European dataset to increase data diversity.

We hypothesize that combining German and English will outperform either language alone, as the model sees more diverse Germanic data while retaining the typological proximity to

Norwegian.

2.3.2 Results

Training set	en	de	zh	ru	es	no
de + en	0.5616	0.6391	0.3564	0.4191	0.4927	0.5151
de + en + ru	0.5935	0.6639	0.4952	0.4743	0.5264	0.5245
Best B1 (de)	0.5941	0.6240	0.1809	0.4043	0.4976	0.4356

Table 6: Krippendorff’s α on all validation sets for B2 multilingual models, compared to best B1 result.

Both multilingual training sets substantially outperform the best single-language model from B1 on Norwegian. Training on de + en improves Norwegian α from 0.4356 to 0.5151, and adding Russian (de + en + ru) pushes it further to 0.5245. This confirms our hypothesis that combining Germanic languages improves transfer to Norwegian.

The addition of Russian is somewhat surprising — Russian is a Slavic language with no close relation to Norwegian, yet it consistently improves performance across nearly all evaluation languages. This is likely due to the increased dataset size rather than linguistic proximity, as adding Russian roughly doubles the training data. The model appears to benefit from seeing more diverse semantic judgments across languages, even when the source language is typologically distant.

Our language selection strategy worked as expected: prioritizing Germanic languages (German and English) gave the best baseline, and adding a large non-Germanic dataset as supplementary data provided further gains. If we were to iterate further, we would try adding all available languages to maximize data volume, and potentially weight the Germanic languages more heavily during training.

3 Task C: Test Predictions

3.1 Best Model Configuration

We decided to go with mmBERT with Target Word Pooling and squared EMD loss on full distribution because this configuration performed the best in B2. We also added Spanish to increase the overall data diversity and training size.

Use of AI Tools

ChatGPT was used as a writing assistant to help structure and formulate parts of this report.

Repository

The code for this assignment is available at:

<https://github.uio.no/sigurnor/in5550-research>

References

- [1] Anssi Eronen et al. Zero-shot cross-lingual transfer language selection using linguistic similarity. *Information Processing & Management*, 60(2), 2023.
 - [2] Wikipedia contributors. Germanic languages, 2024. URL https://en.wikipedia.org/wiki/Germanic_languages.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Marone, M., Weller, O., Fleshman, W., Yang, E., Lawrie, D., & Van Durme, B. (2025, September 9). mmBERT: ModernBERT goes multilingual. Hugging Face Blog. <https://huggingface.co/blog/mmbert>